

Base de datos

```
1  """
2  database.py
3  Maneja la conexión a SQLite y todas las operaciones sobre las tablas:
4  usuarios, productos, clientes, ventas, detalle_ventas, config.
5  """
6
7  import sqlite3
8  import os
9  import hashlib
10 import secrets
11 from datetime import datetime
12
13 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
14 DB_PATH = os.path.join(BASE_DIR, "bodegon.db")
15
16
17 def get_connection():
18     conn = sqlite3.connect(DB_PATH)
19     conn.row_factory = sqlite3.Row
20     conn.execute("PRAGMA foreign_keys = ON")
21     return conn
22
23
24 def hash_password(password: str, salt: str = None):
25     if salt is None:
26         salt = secrets.token_hex(8)
27     h = hashlib.sha256((salt + password).encode("utf-8")).hexdigest()
28     return h, salt
29
30
31 def verificar_password(password: str, hash_guardado: str, salt: str) -> bool:
32     h, _ = hash_password(password, salt)
33     return h == hash_guardado
34
35
36 def init_db():
37     """Crea las tablas si no existen y siembra datos por defecto."""
38     conn = get_connection()
39     cur = conn.cursor()
40
41     cur.execute("""
42         CREATE TABLE IF NOT EXISTS usuarios (
43             id INTEGER PRIMARY KEY AUTOINCREMENT,
44             nombre TEXT NOT NULL,
45             apellido TEXT,
46             email TEXT,
47             usuario TEXT NOT NULL UNIQUE,
48             password_hash TEXT NOT NULL,
```

Ln 20, Co

```
Untitled-1.ipynb • Welcome database.py X
C: > Users > Admin > Desktop > bodegon_pos > database.py > get_connection

36 def init_db():
49     salt TEXT NOT NULL,
50     rol TEXT NOT NULL CHECK(rol IN ('admin', 'cajero')),
51     activo INTEGER NOT NULL DEFAULT 1,
52     intentos_fallidos INTEGER NOT NULL DEFAULT 0,
53     bloqueado_hasta TEXT
54 )
55 """
56
57 cur.execute("""
58     CREATE TABLE IF NOT EXISTS productos (
59         id INTEGER PRIMARY KEY AUTOINCREMENT,
60         nombre TEXT NOT NULL,
61         categoria TEXT,
62         precio_usd REAL NOT NULL DEFAULT 0,
63         stock INTEGER NOT NULL DEFAULT 0,
64         stock_minimo INTEGER NOT NULL DEFAULT 5,
65         activo INTEGER NOT NULL DEFAULT 1
66     )
67 """
68
69 cur.execute("""
70     CREATE TABLE IF NOT EXISTS clientes (
71         id INTEGER PRIMARY KEY AUTOINCREMENT,
72         nombre TEXT NOT NULL,
73         telefono TEXT,
74         cedula TEXT UNIQUE,
75         frecuente INTEGER NOT NULL DEFAULT 0,
76         fecha_registro TEXT NOT NULL
77     )
78 """
79
80 cur.execute("""
81     CREATE TABLE IF NOT EXISTS ventas (
82         id INTEGER PRIMARY KEY AUTOINCREMENT,
83         fecha_hora TEXT NOT NULL,
84         cliente_id INTEGER,
85         usuario_id INTEGER NOT NULL,
86         total_usd REAL NOT NULL,
87         total_bs REAL NOT NULL,
88         tasa_bcv REAL NOT NULL,
89         forma_pago TEXT NOT NULL,
90         anulada INTEGER NOT NULL DEFAULT 0,
91         FOREIGN KEY (cliente_id) REFERENCES clientes(id),
92         FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
93     )
94 """
95
96 cur.execute("""
```

```

97     CREATE TABLE IF NOT EXISTS detalle_ventas (
98         id INTEGER PRIMARY KEY AUTOINCREMENT,
99         venta_id INTEGER NOT NULL,
100        producto_id INTEGER NOT NULL,
101        nombre_producto TEXT NOT NULL,
102        cantidad INTEGER NOT NULL,
103        precio_unitario_usd REAL NOT NULL,
104        FOREIGN KEY (venta_id) REFERENCES ventas(id),
105        FOREIGN KEY (producto_id) REFERENCES productos(id)
106    )
107    """
108
109    cur.execute("""
110        CREATE TABLE IF NOT EXISTS config (
111            clave TEXT PRIMARY KEY,
112            valor TEXT
113        )
114    """)
115
116    cur.execute("""
117        CREATE TABLE IF NOT EXISTS movimientos (
118            id INTEGER PRIMARY KEY AUTOINCREMENT,
119            fecha_hora TEXT NOT NULL,
120            tipo TEXT NOT NULL,
121            descripcion TEXT NOT NULL,
122            usuario_id INTEGER
123        )
124    """)
125
126    conn.commit()
127
128    # Semilla: usuario admin por defecto si no hay ningún usuario
129    cur.execute("SELECT COUNT(*) AS c FROM usuarios")
130    if cur.fetchone()["c"] == 0:
131        crear_usuario(conn, "Administrador", "General", "admin@bodegon.local",
132            "admin", "admin123", "admin")
133        crear_usuario(conn, "Cajero", "Uno", "cajero@bodegon.local",
134            "cajero", "cajero123", "cajero")
135
136    # Semilla: config por defecto (tasas: BCV dólar auto, BCV euro auto, y manual)
137    cur.execute("SELECT COUNT(*) AS c FROM config")
138    if cur.fetchone()["c"] == 0:
139        valores_iniciales = [
140            ("tasa_bcv_usd", "0"),
141            ("tasa_bcv_usd_actualizada", ""),
142            ("tasa_bcv_eur", "0"),
143            ("tasa_bcv_eur_actualizada", ""),

```

```

144         ("tasa_manual", "0"),
145         ("tasa_manual_actualizada", ""),
146         ("tasa_para_ventas", "bcv_usd"), # 'bcv_usd' o 'manual'
147     ]
148     cur.executemany("INSERT INTO config (clave, valor) VALUES (?, ?)", valores_iniciales)
149     conn.commit()
150
151     # Semilla: un par de productos de ejemplo si la tabla está vacía
152     cur.execute("SELECT COUNT(*) AS c FROM productos")
153     if cur.fetchone()["c"] == 0:
154         ejemplo = [
155             ("Harina PAN 1kg", "Alimentos", 1.20, 30, 10),
156             ("Coca Cola 2L", "Bebidas", 1.80, 24, 6),
157             ("Papel Higiénico x4", "Aseo", 2.50, 15, 5),
158             ("Aceite Vatel 1L", "Alimentos", 3.00, 12, 4),
159         ]
160         cur.executemany(
161             "INSERT INTO productos (nombre, categoria, precio_usd, stock, stock_minimo) "
162             "VALUES (?, ?, ?, ?, ?)", ejemplo
163         )
164         conn.commit()
165
166     conn.close()
167
168
169     # ----- USUARIOS
170
171     def crear_usuario(conn, nombre, apellido, email, usuario, password, rol):
172         h, salt = hash_password(password)
173         conn.execute(
174             "INSERT INTO usuarios (nombre, apellido, email, usuario, password_hash, salt, rol) "
175             "VALUES (?, ?, ?, ?, ?, ?, ?)",
176             (nombre, apellido, email, usuario, h, salt, rol)
177         )
178         conn.commit()
179
180
181     def obtener_usuario_por_login(conn, usuario):
182         cur = conn.execute("SELECT * FROM usuarios WHERE usuario = ?", (usuario,))
183         return cur.fetchone()
184
185
186     def listar_usuarios(conn):
187         return conn.execute("SELECT * FROM usuarios ORDER BY id").fetchall()
188
189
190     def eliminar_usuario(conn, usuario_id, realizado_por=None):
191         cur = conn.execute("UPDATE usuarios SET activo = 0 WHERE id = ?", (usuario_id,))

```

```

193 |     registrar_movimiento(conn, "usuario", f"Usuario id {usuario_id} desactivado", realizado_por)
194 |
195 |
196 | def registrar_intento_fallido(conn, usuario_id, intentos, bloqueado_hasta):
197 |     conn.execute(
198 |         "UPDATE usuarios SET intentos_fallidos = ?, bloqueado_hasta = ? WHERE id = ?",
199 |         (intentos, bloqueado_hasta, usuario_id)
200 |     )
201 |     conn.commit()
202 |
203 |
204 | def resetear_intentos(conn, usuario_id):
205 |     conn.execute(
206 |         "UPDATE usuarios SET intentos_fallidos = 0, bloqueado_hasta = NULL WHERE id = ?",
207 |         (usuario_id,)
208 |     )
209 |     conn.commit()
210 |
211 |
212 | def cambiar_password(conn, usuario_id, nueva_password):
213 |     h, salt = hash_password(nueva_password)
214 |     conn.execute(
215 |         "UPDATE usuarios SET password_hash = ?, salt = ? WHERE id = ?",
216 |         (h, salt, usuario_id)
217 |     )
218 |     conn.commit()
219 |
220 |
221 | # ----- PRODUCTOS
222 |
223 | def listar_productos(conn, solo_activos=True):
224 |     if solo_activos:
225 |         return conn.execute(
226 |             "SELECT * FROM productos WHERE activo = 1 ORDER BY nombre"
227 |         ).fetchall()
228 |     return conn.execute("SELECT * FROM productos ORDER BY nombre").fetchall()
229 |
230 |
231 | def productos_bajo_stock(conn):
232 |     return conn.execute(
233 |         "SELECT * FROM productos WHERE activo = 1 AND stock <= stock_minimo ORDER BY stock"
234 |     ).fetchall()
235 |
236 |
237 | def agregar_producto(conn, nombre, categoria, precio_usd, stock, stock_minimo, usuario_id=None):
238 |     conn.execute(
239 |         "INSERT INTO productos (nombre, categoria, precio_usd, stock, stock_minimo) "
240 |         "VALUES (?, ?, ?, ?, ?)",
241 |         (nombre, categoria, precio_usd, stock, stock_minimo)

```

```

242     )
243     conn.commit()
244     registrar_movimiento(conn, "producto", f"Producto agregado: {nombre} (stock inicial {stock})", usuario_id)
245
246
247 def editar_producto(conn, producto_id, nombre, categoria, precio_usd, stock_minimo, usuario_id=None):
248     conn.execute(
249         "UPDATE productos SET nombre=?, categoria=?, precio_usd=?, stock_minimo=? WHERE id=?",
250         (nombre, categoria, precio_usd, stock_minimo, producto_id)
251     )
252     conn.commit()
253     registrar_movimiento(conn, "producto", f"Producto editado: {nombre} (precio ${precio_usd:.2f})", usuario_id)
254
255
256 def ajustar_stock(conn, producto_id, cantidad_delta, usuario_id=None):
257     conn.execute(
258         "UPDATE productos SET stock = stock + ? WHERE id = ?",
259         (cantidad_delta, producto_id)
260     )
261     conn.commit()
262     producto = obtener_producto(conn, producto_id)
263     signo = "+" if cantidad_delta >= 0 else ""
264     registrar_movimiento(
265         conn, "stock",
266         f"Ajuste de stock en '{producto['nombre']}': {signo}{cantidad_delta} (nuevo stock: {producto['stock']})",
267         usuario_id
268     )
269
270
271 def eliminar_producto(conn, producto_id, usuario_id=None):
272     producto = obtener_producto(conn, producto_id)
273     conn.execute("UPDATE productos SET activo = 0 WHERE id = ?", (producto_id,))
274     conn.commit()
275     registrar_movimiento(conn, "producto", f"Producto eliminado: {producto['nombre']}", usuario_id)
276
277
278 def obtener_producto(conn, producto_id):
279     return conn.execute("SELECT * FROM productos WHERE id = ?", (producto_id,)).fetchone()
280
281
282 # ----- CLIENTES
283
284 def buscar_cliente_por_cedula(conn, cedula):
285     return conn.execute("SELECT * FROM clientes WHERE cedula = ?", (cedula,)).fetchone()
286
287
288 def buscar_clientes_por_nombre(conn, texto):

```

C:\Users\Admin\Desktop> bodegon_pos > database.py > get_connection

```
287
288 def buscar_clientes_por_nombre(conn, texto):
289     return conn.execute(
290         "SELECT * FROM clientes WHERE nombre LIKE ? ORDER BY nombre",
291         (f"%{texto}%",)
292     ).fetchall()
293
294
295 def registrar_cliente(conn, nombre, telefono, cedula, frecuente=False):
296     conn.execute(
297         "INSERT INTO clientes (nombre, telefono, cedula, frecuente, fecha_registro) "
298         "VALUES (?, ?, ?, ?, ?)",
299         (nombre, telefono, cedula, 1 if frecuente else 0, datetime.now().isoformat())
300     )
301     conn.commit()
302
303
304 def listar_clientes(conn):
305     return conn.execute("SELECT * FROM clientes ORDER BY nombre").fetchall()
306
307
308 # ----- VENTAS
309
310 def registrar_venta(conn, cliente_id, usuario_id, carrito, tasa_bcv, forma_pago):
311     """
312     carrito: lista de dicts {producto_id, nombre, cantidad, precio_unitario_usd}
313     Descuenta stock y guarda venta + detalle. Devuelve el id de la venta.
314     """
315     total_usd = sum(item["cantidad"] * item["precio_unitario_usd"] for item in carrito)
316     total_bs = total_usd * tasa_bcv
317
318     cur = conn.cursor()
319     cur.execute(
320         "INSERT INTO ventas (fecha_hora, cliente_id, usuario_id, total_usd, total_bs, tasa_bcv, forma_pago) "
321         "VALUES (?, ?, ?, ?, ?, ?, ?)",
322         (datetime.now().isoformat(), cliente_id, usuario_id, total_usd, total_bs, tasa_bcv, forma_pago)
323     )
324     venta_id = cur.lastrowid
325
326     for item in carrito:
327         cur.execute(
328             "INSERT INTO detalle_ventas (venta_id, producto_id, nombre_producto, cantidad, precio_unitario_usd) "
329             "VALUES (?, ?, ?, ?, ?)",
330             (venta_id, item["producto_id"], item["nombre"], item["cantidad"], item["precio_unitario_usd"])
331         )
332         cur.execute(
333             "UPDATE productos SET stock = stock - ? WHERE id = ?",
334             (item["cantidad"], item["producto_id"])
335         )
336     \
```

C:\Users\Admin\Desktop> bodegon_pos > database.py > get_connection

```
330 def registrar_venta(conn, cliente_id, usuario_id, carrito, tasa_bcv, forma_pago):
331     """
332     """
333     "UPDATE productos SET stock = stock - ? WHERE id = ?",
334     (item["cantidad"], item["producto_id"])
335     )
336
337     conn.commit()
338     registrar_movimiento(
339         conn, "venta",
340         f"Venta #{venta_id} por ${total_usd:.2f} (Bs {total_bs:.2f}) - {forma_pago}",
341         usuario_id
342     )
343     return venta_id
344
345
346 def obtener_venta(conn, venta_id):
347     return conn.execute("SELECT * FROM ventas WHERE id = ?", (venta_id,)).fetchone()
348
349
350 def obtener_detalle_venta(conn, venta_id):
351     return conn.execute(
352         "SELECT * FROM detalle_ventas WHERE venta_id = ?", (venta_id,)
353     ).fetchall()
354
355
356 def ventas_entre_fechas(conn, fecha_ini_iso, fecha_fin_iso):
357     return conn.execute(
358         "SELECT * FROM ventas WHERE fecha_hora BETWEEN ? AND ? AND anulada = 0 ORDER BY fecha_hora DESC",
359         (fecha_ini_iso, fecha_fin_iso)
360     ).fetchall()
361
362
363 def productos_mas_vendidos(conn, fecha_ini_iso, fecha_fin_iso, limite=10):
364     return conn.execute("""
365         SELECT dv.nombre_producto, SUM(dv.cantidad) AS unidades,
366         SUM(dv.cantidad * dv.precio_unitario_usd) AS total_usd
367         FROM detalle_ventas dv
368         JOIN ventas v ON v.id = dv.venta_id
369         WHERE v.fecha_hora BETWEEN ? AND ? AND v.anulada = 0
370         GROUP BY dv.producto_id
371         ORDER BY unidades DESC
372         LIMIT ?
373     """, (fecha_ini_iso, fecha_fin_iso, limite)).fetchall()
374
375
376 # ----- CONFIG
377
378 def get_config(conn, clave, default=None):
379     row = conn.execute("SELECT valor FROM config WHERE clave = ?", (clave,)).fetchone()
```

```

C:\Users\Admin\Desktop\bodegon_pos > database.py > get_connection
377
378 def get_config(conn, clave, default=None):
379     row = conn.execute("SELECT valor FROM config WHERE clave = ?", (clave,)).fetchone()
380     return row["valor"] if row else default
381
382
383 def set_config(conn, clave, valor):
384     conn.execute(
385         "INSERT INTO config (clave, valor) VALUES (?, ?) "
386         "ON CONFLICT(clave) DO UPDATE SET valor = excluded.valor",
387         (clave, str(valor))
388     )
389     conn.commit()
390
391
392 # ----- MOVIMIENTOS
393
394 def registrar_movimiento(conn, tipo, descripcion, usuario_id=None):
395     """
396     Guarda un renglón en la bitácora de movimientos (para el registro diario).
397     tipo típico: 'venta', 'producto', 'stock', 'usuario', 'tasa'.
398     """
399     conn.execute(
400         "INSERT INTO movimientos (fecha_hora, tipo, descripcion, usuario_id) VALUES (?, ?, ?, ?)",
401         (datetime.now().isoformat(), tipo, descripcion, usuario_id)
402     )
403     conn.commit()
404
405
406 def movimientos_entre_fechas(conn, fecha_ini_iso, fecha_fin_iso):
407     return conn.execute(
408         "SELECT m.*, u.nombre AS usuario_nombre FROM movimientos m "
409         "LEFT JOIN usuarios u ON u.id = m.usuario_id "
410         "WHERE m.fecha_hora BETWEEN ? AND ? ORDER BY m.fecha_hora",
411         (fecha_ini_iso, fecha_fin_iso)
412     ).fetchall()
413

```